AI-Powered Escape Room

Version 1 Architecture: the LLM as Game Master

Streamlit interface | Python app | Google Gemini API

System architecture

1. User interface (Streamlit)

- Chat box for natural-language actions
- Sidebar with instructions, inventory and a Restart button
- Shows the Game Master's replies

2. Application logic (Python, app_v1.py)

- Keeps messages, inventory and the escaped flag in session state
- Sends the system prompt and full chat history to Gemini
- Reads hidden tags to update the inventory and detect the escape

3. Game Master (Google Gemini)

- The system prompt holds the room, puzzles and codes
- Decides what succeeds and writes the narration
- Adds [ITEM] and [ESCAPED] tags to its replies

Reply text with hidden [ITEM: …] / [ESCAPED] tags returns to the app, which strips the tags and shows the reply

The API key is read from .streamlit/secrets.toml and is never stored in the code.

What happens in one turn

1

Player types

A natural-language action, e.g. “examine the bookshelf”

2

App stores it

The message is appended to the chat history in session state

3

Gemini replies

It receives the system prompt and the whole history and answers as Game Master

4

Tags are read

[ITEM: …] updates the inventory; [ESCAPED] marks the game as won

5

Player sees it

The tags are hidden and the reply appears in the chat

The chat history is the game's only memory: nothing else records what the player has done.

Example playthrough

1. Investigate

PLAYER

“I examine the bookshelf.”

OUTCOME

Four books show the numbers 4, 1, 7, 3.

2. Open the drawer

PLAYER

“I try 4173 on the drawer.”

OUTCOME

The drawer opens and a brass key goes into the inventory.

3. Use the key

PLAYER

“I use the brass key on the box behind the painting.”

OUTCOME

A note about the stopped clock is found.

4. Escape

PLAYER

“I enter 0947 on the door.”

OUTCOME

The clock shows 9:47, the door opens and [ESCAPED] ends the game.

Why Version 1 is not reliable enough

Memory is only chat history

The LLM can forget or contradict earlier actions.

Solutions are in the prompt

A player can ask for the codes and the LLM may reveal them.

The LLM decides what succeeds

Missing keys, wrong codes or false claims may be accepted.

State depends on tags

If the LLM forgets a tag, the inventory or the win is wrong.

Version 2 moves state, rules and solutions into Python and lets the LLM only narrate.